

Cálculo Diferencial Aplicado a la Gestión

Tecnicatura Universitaria en Análisis y Gestión de Datos

Manuel Fernández San Martín

Laboratorio de Métodos Cuantitativos Aplicados a la Gestión
Universidad de Buenos Aires

Primer cuatrimestre 2026

Calcular, interpretar y aplicar la derivada de funciones económicas para analizar variaciones, marginalidad y elasticidades.

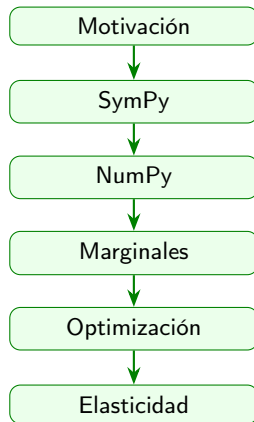
¿Qué vamos a ver hoy?

Hoja de ruta

- 1 **Motivación:** ¿para qué sirve derivar en gestión?
- 2 **Derivada simbólica** con SymPy
- 3 **Derivada numérica** con NumPy
- 4 **Funciones marginales:** MR , MC , MB
- 5 **Optimización:** regla $MR = MC$
- 6 **Elasticidad-precio** de la demanda

Herramientas del día

SymPy (álgebra simbólica) NumPy (cálculo numérico)
Matplotlib (gráficos)



Motivación

¿Por qué necesitamos la derivada?

La paradoja del valor: Agua vs. Diamantes

El planteo clásico (Adam Smith)

¿Por qué el **agua**, que es esencial para la supervivencia, es tan barata, mientras que los **diamantes**, que son un mero adorno, son tan caros?

El callejón sin salida

Durante décadas, la economía intentó explicar los precios basándose en la **utilidad total** del bien o en el costo histórico de producirlo. Ninguna explicaba la paradoja.

Agua

Muy abundante
Utilidad Total: Incalculable
Precio: Casi nulo

Diamantes

Muy escasos
Utilidad Total: Baja
Precio: Altísimo

El misterio se mantuvo sin resolver durante casi un siglo. . .

La solución: La Revolución Marginalista (1870s)

El cambio de paradigma

El valor **no** depende de la utilidad total, sino de la **utilidad marginal**: la satisfacción que aporta la *última unidad* consumida.

Los pioneros

- W. Stanley **Jevons** (Inglaterra)
- Carl **Menger** (Austria)
- Léon **Walras** (Suiza/Francia)



Resolviendo la paradoja

- **Agua**: Como es tan abundante, una gota extra (marginal) casi no nos aporta valor adicional.
- **Diamantes**: Al ser tan escasos, conseguir uno extra genera muchísima utilidad.

$$\text{Valor} \propto \frac{\Delta(\text{Utilidad})}{\Delta(\text{Cantidad})}$$

¿A qué concepto matemático se parece esto?

Hicks y el triunfo del cálculo diferencial



La formalización (1930s)

John **Hicks** (y R.G.D. Allen) sentaron las bases de la microeconomía moderna en su obra *Value and Capital* (1939).

- Abandonaron la idea de medir la utilidad con números absolutos.
- Introdujeron las **curvas de indiferencia** modernas.
- Demostraron que lo único que importa son las **tasas marginales de sustitución**.

El lenguaje de la gestión

Hicks ayudó a demostrar que **toda decisión económica racional** se basa en comparar variables en el margen (costo vs. beneficio de una unidad extra).

Marginalidad = Derivada

$$\text{Marginal} = \lim_{\Delta q \rightarrow 0} \frac{\Delta f}{\Delta q} = \frac{df}{dq}$$

Entender el cálculo es entender cómo se toman decisiones bajo escasez.

Una pregunta de gestión

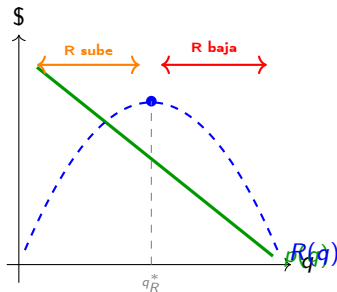
Escenario

Una empresa fija el precio de su producto y observa:

- A \$100 vende 500 unidades/mes
- A \$105 vende 470 unidades/mes

Preguntas clave

- Subir el precio ¿**aumenta** o **baja** el ingreso?
- ¿Cuántas unidades producir para **maximizar** el beneficio?
- ¿A partir de qué punto la producción se vuelve **ineficiente**?



La **derivada**

nos dice si la función **sube** o **baja** en cada punto y a qué velocidad.

La derivada como herramienta de decisión

¿Qué mide la derivada?

La **tasa de variación instantánea**:

$$f'(x) = \lim_{\Delta x \rightarrow 0} \frac{f(x + \Delta x) - f(x)}{\Delta x}$$

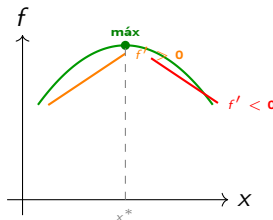
Geométricamente: **pendiente de la tangente**.

Interpretación del signo

$f'(x) > 0$	función creciente
$f'(x) < 0$	función decreciente
$f'(x) = 0$	candidato a óptimo

En gestión, la derivada responde...

- ¿Estoy en zona de **ingresos crecientes**?
- ¿Cuál es el precio que **maximiza** el beneficio?
- ¿Cuánto cambia la demanda si **subo el precio 1%**?



Dos maneras de derivar en Python

Derivada **simbólica**

- Conozco la **fórmula** de $f(x)$
- Resultado: **expresión exacta** $f'(x)$
- Herramienta: SymPy

Ejemplo: modelé la demanda como
 $p(q) = 160 - 6q + 0,8 \ln(1 + q)$ y quiero calcular
 $MR(q) = \frac{dR}{dq}$

Derivada **numérica**

- Solo tengo **datos observados**
- Resultado: **aproximación discreta**
- Herramienta: NumPy

Ejemplo: tengo precios diarios de una acción y quiero detectar **tendencias de crecimiento**

En la práctica, se combinan

Se modela con SymPy para obtener la función marginal **teórica**, y se valida con `np.gradient` sobre **datos reales**.

Sección 1

La derivada simbólica con SymPy

sp.diff: derivar en una línea

Sintaxis básica: sp.diff(f, x, n)

- f: expresión a derivar
- x: variable de diferenciación
- n: orden (por defecto 1)

```
import sympy as sp
x = sp.symbols("x", real=True)

f = x**2
sp.diff(f, x)      # -> 2*x

g = sp.sin(x)
sp.diff(g)         # -> cos(x)
sp.diff(g, x, 2)   # -> -sin(x)
sp.diff(g, x, 5)   # -> cos(x)
```

Nota

Para funciones univariables, x puede omitirse. Para multivariables es **obligatorio**.

Reglas que SymPy aplica automáticamente

$$f = x^n \quad f' = nx^{n-1}$$

$$f = e^x \quad f' = e^x$$

$$f = \ln x \quad f' = 1/x$$

$$f = \sin x \quad f' = \cos x$$

$$f = \cos x \quad f' = -\sin x$$

$$(f \cdot g)' \quad f'g + fg'$$

Evaluar la derivada en un punto

Sea $q(x) = 24x^4 + 12x^2 - 6$.

```
q      = 24*x**4 + 12*x**2 - 6
dq     = sp.diff(q, x)      # 96x^3+24x
d2q    = sp.diff(q, x, 2)  # 288x^2+24

# Evaluar en x = 6
dq.subs(x, 6).evalf()     # 20952.0
d2q.subs(x, 6).evalf()    # 10392.0
```

`.subs()` y `.evalf()`

- `.subs(x, val)`: reemplaza la variable
- `.evalf()`: convierte a float

Verificación manual

$$q'(x) = 96x^3 + 24x$$

$$q'(6) = 96 \cdot 216 + 24 \cdot 6 = 20,952 \checkmark$$

¿Cuándo usar evaluación puntual?

- Calcular *MR* o *MC* en un nivel de producción específico
- Verificar condiciones de optimización
- Comparar resultados teóricos con datos observados

sp.lambdify: del símbolo al gráfico (1/2)

¿Para qué sirve?

Convierte una expresión SymPy en una **función numérica** evaluable con NumPy. Sin él, habría que llamar `.subs()` punto por punto.

```
x_vals = np.linspace(-2, 2, 500)
q_f = sp.lambdify(x, q)           # funcion original
dqdx_f = sp.lambdify(x, dq)       # primera derivada
d2qdx2_f = sp.lambdify(x, d2q)    # segunda derivada

plt.plot(x_vals, q_f(x_vals),      label="q")
plt.plot(x_vals, dqdx_f(x_vals),   label="q'")
plt.plot(x_vals, d2qdx2_f(x_vals), label="q''")
plt.axhline(0, color='black', lw=0.8)
plt.legend(); plt.grid(True); plt.show()
```

Flujo completo

SymPy → sp.diff → sp.lambdify → NumPy → Matplotlib

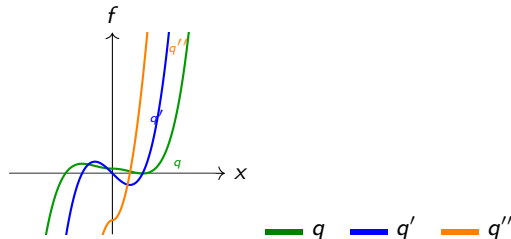
sp.lambdify: del símbolo al gráfico (2/2)

¿Qué vemos en el gráfico?

El resultado de lambdificar y graficar las tres derivadas de $q(x) = 24x^4 + 12x^2 - 6$:

Lectura del gráfico

- q crece donde $q' > 0$
- q tiene extremos donde $q' = 0$
- $q'' < 0$ indica **máximos locales**
- $q'' > 0$ indica **mínimos locales**



Para recordar

`lambdify` es el puente entre el álgebra simbólica y los gráficos.

sp.simplify: limpiar el resultado

¿Cuándo usarlo?

Cuando SymPy devuelve una expresión con **redundancias algebraicas**.

```
r = ((x**2 + 2*x + 1)
      * sp.exp(sp.log(x**2))
      + (sp.sin(x)**2
          + sp.cos(x)**2)/x)

drdx = sp.diff(r, x)
# -> expresion larga

sp.simplify(drdx)
# -> 4*x**3 + 6*x**2 + 2*x - 1/x**2
```

Identidades que SymPy reconoce

- $x^2 + 2x + 1 = (x + 1)^2$
- $e^{\ln(x^2)} = x^2 \quad (x > 0)$
- $\sin^2(x) + \cos^2(x) = 1$

Recomendación práctica

Aplicar `sp.simplify` **después** de derivar para obtener expresiones más legibles e interpretables.

Sección 2

La derivada numérica con NumPy

El problema: datos sin fórmula

Situación real

En muchos contextos de gestión **no** conocemos la función que generó los datos:

- Precios históricos de acciones
- Ventas diarias de un producto
- Tráfico web por hora
- Demanda observada en el mercado

El desafío

Queremos saber si la variable **crece o decrece** y cuándo cambia de tendencia — pero sin una fórmula.



La derivada

numérica aproxima f' directamente a partir de los valores observados.

np.gradient: gradiente discreto

Idea central

Usa **diferencias finitas centradas**:

$$f'(x_i) \approx \frac{f(x_{i+1}) - f(x_{i-1}))}{x_{i+1} - x_{i-1}}$$

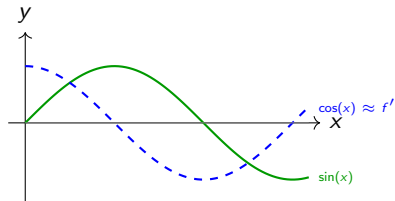
Devuelve un array de **la misma longitud** que los datos.

```
x = np.linspace(0, 10, 50)
y_smooth = np.sin(x)

# Derivada numerica
dy_dx = np.gradient(y_smooth, x)
```

Problema con ruido

Si los datos tienen **ruido**, np.gradient **amplifica** ese ruido. El resultado es ilegible.



Con datos suaves: funciona bien

La derivada numérica de sin(x) recupera cos(x) con buena precisión.

Primero suavizar, luego derivar

Estrategia en 2 pasos

- 1 Aplicar un **promedio móvil** con `np.convolve`
- 2 Calcular `np.gradient` sobre los datos suavizados

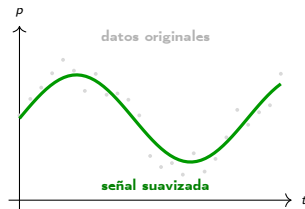
```
# Ventana de 15 días (area = 1)
rect = np.ones(15) / 15

# Convolucion = promedio movil
y_smooth = np.convolve(
    datos["precio"], rect, mode="valid")

# Gradiente sobre datos suavizados
dy_smooth = np.gradient(y_smooth, x[7:-7])
```

Regla práctica

Nunca aplicar `np.gradient` directamente a series temporales con alta volatilidad.



Aplicaciones

Precios de acciones, ventas diarias, tráfico web, etc.

Simbólica vs. Numérica: ¿cuándo usar cada una?

	Simbólica (SymPy)	Numérica (NumPy)
Entrada	Fórmula algebraica	Array de valores observados
Resultado	Expresión exacta $f'(x)$	Array de aproximaciones $f'(x_i)$
Precisión	Exacta (analítica)	Aproximada (dif. finitas)
Ruido	No aplica	Se amplifica; requiere suavizado
Uso típico	Modelos teóricos	Datos reales, series de tiempo
Herramienta	<code>sp.diff</code> , <code>sp.lambdify</code>	<code>np.gradient</code> , <code>np.convolve</code>

Para datos organizacionales...

En la práctica se **combinan**: modelar con SymPy para la función marginal teórica, y validar con `np.gradient` sobre datos reales.

Sección 3

Funciones organizacionales y funciones marginales

¿Qué es una función marginal?

Idea central

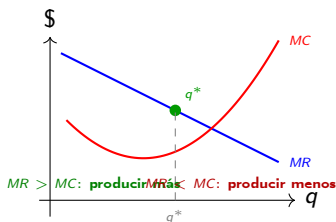
Una función marginal mide cuánto **cambia** una magnitud (ingreso, costo, beneficio) cuando la producción aumenta en **una unidad**.

Las tres funciones marginales

$$MR(q) = \frac{dR(q)}{dq} \quad (\text{ingreso marginal})$$

$$MC(q) = \frac{dC(q)}{dq} \quad (\text{costo marginal})$$

$$MB(q) = \frac{dB(q)}{dq} = MR - MC$$



Regla de optimización

El beneficio se **maximiza** donde:

$$MR(q^*) = MC(q^*)$$

Modelo simbólico: demanda y costos

Estructura del modelo

Demanda inversa: $p(q) = a - bq + c \ln(1 + q)$

Ingreso: $R(q) = p(q) \cdot q$

Costo: $C(q) = F + c_0q - c_1q^2 + c_2q^3 + c_3\sqrt{q}$

Beneficio: $B(q) = R(q) - C(q)$

```
q = sp.symbols('q', real=True)
# ... definir a, b, c, F, c0, c1, c2, c3

p = a - b*q + c*sp.log(1+q)
R = p * q
C = F + c0*q - c1*q**2 + c2*q**3 + c3*sp.sqrt(q)
B = R - C

MR = sp.diff(R, q)
MC = sp.diff(C, q)
MB = sp.diff(B, q)
```

Parámetros del ejemplo

$a = 160$, $b = 6$, $c = 0,8$

$F = 20$, $c_0 = 12$, $c_1 = 2,5$

$c_2 = 0,15$, $c_3 = 3$

Flujo recomendado

- 1 Definir con símbolos genéricos
- 2 `.subs(params)`: parámetros
- 3 `sp.lambdify`: a función numérica
- 4 Evaluar sobre `np.linspace`
- 5 Graficar e interpretar

Análisis del ingreso total y marginal

¿Qué buscamos?

El punto donde $MR(q) = 0$: el nivel de producción que **maximiza el ingreso**.

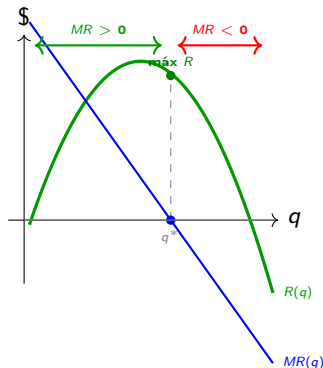
Resultado numérico

$MR = 0$ en $q^* \approx 13,07$

$R''(q^*) < 0 \Rightarrow$ **máximo de R**

Interpretación

- $q < 13,07$: $MR > 0$, conviene **vender más**
- $q > 13,07$: $MR < 0$, vender más **reduce el ingreso**



Segunda derivada

$f''(x^*) < 0 \Rightarrow$ **máximo**

$f''(x^*) > 0 \Rightarrow$ **mínimo**

Análisis del costo marginal

¿Qué buscamos?

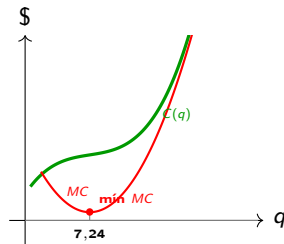
El **mínimo del costo marginal**: la escala de producción más eficiente. A partir de ese punto, cada unidad extra es más cara.

Dos puntos críticos de MC

q	tipo
3,92	máximo local
7,24	mínimo local

Interpretación

A partir de $q \approx 7,24$ el costo marginal sube: rendimientos decrecientes.



Gestión práctica

El mínimo de MC determina la **capacidad óptima** de la planta o turno de producción.

Análisis del beneficio: regla $MR = MC$

Regla de oro

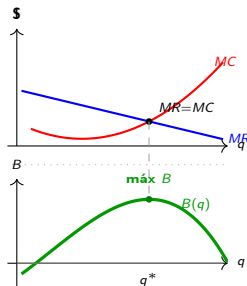
Máximo beneficio donde

$$MR(q^*) = MC(q^*) \Leftrightarrow MB(q^*) = 0$$

con $MB'(q^*) < 0$ (máximo confirmado).

Tres óptimos distintos

Óptimo	q	Condición
R máx	13,07	$MR = 0$
MC mín	7,24	$MC' = 0$
B máx	9,85	$MB = 0 \Leftrightarrow q^*$



Sección 4

Elasticidad-precio de la demanda

Elasticidad: ¿qué mide y para qué sirve?

Definición

$$\varepsilon = \frac{\% \Delta q}{\% \Delta p} = \frac{dq}{dp} \cdot \frac{p}{q}$$

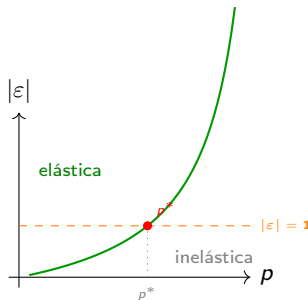
Mide la **sensibilidad** de la demanda ante cambios en el precio.

Interpretación de $|\varepsilon|$

- $|\varepsilon| > 1$: **elástica** — muy sensibles
- $|\varepsilon| = 1$: **unitaria** — ingreso máximo
- $|\varepsilon| < 1$: **inelástica** — poco sensibles

Implicancias para fijar precios

- **Elástica**: subir precio **reduce** el ingreso
- **Inelástica**: subir precio **aumenta** el ingreso
- **Unitaria**: el ingreso es **máximo**



Elasticidad arco vs. elasticidad puntual

Elasticidad arco (datos discretos)

Para dos puntos (p_1, q_1) y (p_2, q_2) :

$$\varepsilon_{arco} = \frac{q_2 - q_1}{p_2 - p_1} \cdot \frac{p_1 + p_2}{q_1 + q_2}$$

Usa el punto medio para evitar que el resultado dependa de la dirección del cambio.

Usar cuando: se tienen observaciones de mercado históricas.

Elasticidad puntual (función conocida)

Si $q = D(p)$:

$$\varepsilon(p) = D'(p) \cdot \frac{p}{D(p)}$$

Para la demanda inversa $p = p(q)$:

$$\varepsilon(q) = \frac{1}{p'(q)} \cdot \frac{p(q)}{q}$$

Usar cuando: se tiene la función de demanda analítica.

Elasticidad y decisiones empresariales

Tipo	$ \varepsilon $	Si se sube el precio	Ejemplos
Perf. inelástico	0	Ingreso sube igual	Insulina, agua
Inelástico	$(0, 1)$	Ingreso sube	Combustible
Unitaria	1	Ingreso no cambia	—
Elástico	$(1, \infty)$	Ingreso baja	Autos, turismo

Aplicación a la gestión

Para fijar precios: si la demanda es inelástica $\rightarrow$ subir precio aumenta ingresos. Si es elástica $\rightarrow$ bajar precio puede aumentar ingresos. La **derivada** es la herramienta que permite calcular esto con precisión.

Ideas clave de la clase

Conceptual

- La derivada mide la **tasa de variación instantánea**: pendiente de la tangente.
- $f'(x^*) = 0$ es condición necesaria del óptimo; la segunda derivada confirma el tipo.
- El **máximo beneficio** se alcanza donde $MR = MC$.
- La **elasticidad-precio** orienta decisiones de fijación de precios.

Técnico

- SymPy: derivadas exactas con `sp.diff`, `sp.lambdify`, `sp.simplify`.
- NumPy: aproximación desde datos con `np.gradient` y `np.convolve`.
- **Primero suavizar, luego derivar** en series con ruido.

Próxima clase

Integrales: costo total a partir del marginal, excedente del consumidor.

Actividad práctica en el Notebook

Contexto

Una empresa tiene $p(q) = 80 - 3q + 0,5 \ln(1 + q)$ y $C(q) = 15 + 8q + 0,1q^3$.

Bloque 1 — Simbólico

- 1 Definir $R(q)$, $C(q)$ y $B(q)$ con SymPy.
- 2 Calcular y simplificar MR , MC y MB .
- 3 Evaluar en $q = 0, 5, 10, 15, 20$.
- 4 Lambdificar y graficar en $q \in [0, 25]$.

Bloque 2 — Numérico y Análisis

- 5 Encontrar q^* que maximiza B . Verificar $MR = MC$.
- 6 Graficar $\varepsilon(q)$. ¿Dónde la demanda es elástica?
- 7 **Extra:** `np.gradient` sobre ventas mensuales.

Entregable: Notebook de Colab con código comentado, gráficos y conclusiones económicas.

Bibliografía

Referencias principales

- Chiang, A. C. & Wainwright, K. (2006). *Fundamental Methods of Mathematical Economics*. McGraw-Hill.
- Varian, H. R. (2010). *Microeconomic Analysis* (3rd ed.). Norton.
- Leithold, L. (1998). *Matemáticas para administración y economía*. Oxford University Press.

Documentación técnica

- SymPy: docs.sympy.org/latest/tutorials/intro-tutorial/calculus.html
- NumPy gradient: numpy.org/doc/stable/reference/generated/numpy.gradient.html
- Repositorio: github.com/espartaca75-prog/LMC-FCE--UBA

¡Gracias!

manuelfsm03@gmail.com

Laboratorio de Métodos Cuantitativos Aplicados a la Gestión — FCE UBA